

Submitted By

Student Name: Supratim Gogoi

Roll Number: CS-BTC24-17

Course / Subject Name

Summer Internship on Cyber Security and Secure App Development

Assignment Report on

Raw Socket Packet Analysis and Protocol Investigation

1. Objective

The objective of this assignment is to understand how raw sockets work in Linux by capturing network packets directly at the network layer, bypassing the usual TCP/UDP abstraction. The assignment involves writing a C program that uses a raw socket to capture packets belonging to the assigned protocol (ICMP), extracting and analysing the IP and ICMP header fields, generating live network traffic to feed the capture, and cross-checking the captured data against Wireshark to confirm that the values reported by the program are accurate.

2. System Configuration

The experiments for this assignment were carried out on the following system:

Component	Configuration
Operating System	Fedora Linux 44 (kernel 7.1.4-200.fc44.x86_64, SMP PREEMPT_DYNAMIC)
Programming Language	C
Compiler	GCC (GNU Compiler Collection) 16.1.1 20260515 (Red Hat 16.1.1-2)
Development Environment	Terminal
Packet Capture Tool	Wireshark
Network Utility Tools	ping
Socket Type	Linux Raw Socket (SOCK_RAW, IPPROTO_ICMP)
Assigned Protocol	ICMP
Network Interface	wlp3s0 (wireless), inet 10.207.165.168/24
Execution Privileges	Root (sudo)
Output Files	program_output.txt, capture.pcapng

```
supratim@fedora:~/Music/Raw$ ping -c 5 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=228 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=250 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=273 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=77.2 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=116 time=84.5 ms

--- 8.8.8.8 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4004ms
rtt min/avg/max/mdev = 77.164/182.439/272.958/84.230 ms
supratim@fedora:~/Music/Raw$ ping -c 25 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=116 time=368 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=116 time=174 ms
64 bytes from 8.8.8.8: icmp_seq=3 ttl=116 time=160 ms
64 bytes from 8.8.8.8: icmp_seq=4 ttl=116 time=84.4 ms
64 bytes from 8.8.8.8: icmp_seq=5 ttl=116 time=277 ms
64 bytes from 8.8.8.8: icmp_seq=6 ttl=116 time=116 ms
64 bytes from 8.8.8.8: icmp_seq=7 ttl=116 time=201 ms
64 bytes from 8.8.8.8: icmp_seq=8 ttl=116 time=117 ms
64 bytes from 8.8.8.8: icmp_seq=9 ttl=116 time=109 ms
64 bytes from 8.8.8.8: icmp_seq=10 ttl=116 time=152 ms
64 bytes from 8.8.8.8: icmp_seq=11 ttl=116 time=94.6 ms
64 bytes from 8.8.8.8: icmp_seq=12 ttl=116 time=106 ms
64 bytes from 8.8.8.8: icmp_seq=13 ttl=116 time=220 ms
64 bytes from 8.8.8.8: icmp_seq=14 ttl=116 time=84.5 ms
64 bytes from 8.8.8.8: icmp_seq=15 ttl=116 time=79.1 ms
64 bytes from 8.8.8.8: icmp_seq=16 ttl=116 time=295 ms
64 bytes from 8.8.8.8: icmp_seq=17 ttl=116 time=317 ms
64 bytes from 8.8.8.8: icmp_seq=18 ttl=116 time=92.5 ms
64 bytes from 8.8.8.8: icmp_seq=19 ttl=116 time=72.7 ms
64 bytes from 8.8.8.8: icmp_seq=20 ttl=116 time=99.7 ms
64 bytes from 8.8.8.8: icmp_seq=21 ttl=116 time=200 ms
64 bytes from 8.8.8.8: icmp_seq=22 ttl=116 time=222 ms
64 bytes from 8.8.8.8: icmp_seq=23 ttl=116 time=244 ms
64 bytes from 8.8.8.8: icmp_seq=24 ttl=116 time=99.2 ms
64 bytes from 8.8.8.8: icmp_seq=25 ttl=116 time=296 ms

--- 8.8.8.8 ping statistics ---
25 packets transmitted, 25 received, 0% packet loss, time 24025ms
rtt min/avg/max/mdev = 72.662/171.209/367.697/85.806 ms
supratim@fedora:~/Music/Raw$
```

Fig. 1 — Host networking details (uname -a, ip addr) and a sample ping to 8.8.8.8

3. Compile and Run Commands

The following commands were used to compile and execute the raw socket packet capture program.

Compile the Program

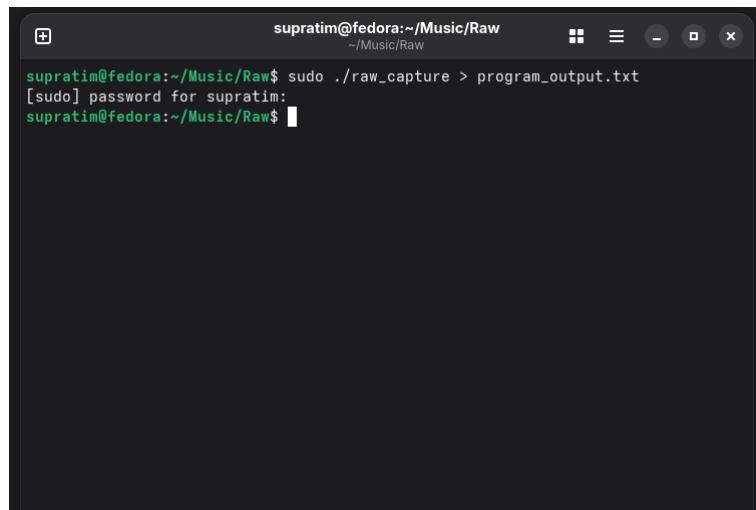
```
gcc raw_capture.c -o raw_capture
```

This compiles raw_capture.c and produces an executable named raw_capture.

Execute the Program

Raw sockets require administrator privileges, so the program was run with sudo and its output was redirected to a file:

```
sudo ./raw_capture > program_output.txt
```

A terminal window titled 'supratim@fedora:~/Music/Raw' with standard window controls. The terminal shows the command 'sudo ./raw_capture > program_output.txt' being executed. It prompts for a password, which is entered as 'supratim'. The prompt then returns to the shell.

```
supratim@fedora:~/Music/Raw$ sudo ./raw_capture > program_output.txt
[sudo] password for supratim:
supratim@fedora:~/Music/Raw$
```

Fig. 2 — Running raw_capture and redirecting its output to program_output.txt

In this implementation the roll number and assigned protocol are set as compile-time constants (ROLL_NO and ASSIGNED_PROTOCOL) and the number of packets to capture is fixed at 20 (MAX_PACKETS). Because of this, the program does not prompt for any input at runtime — it opens the raw socket immediately and starts capturing as soon as it is launched, so traffic generation has to be started separately (in another terminal) once the program is running.

4. Program Design

The program was written in C using a Linux raw socket bound to the ICMP protocol. It listens on the socket for incoming IP packets, filters for ICMP traffic, extracts the relevant header fields, prints them to standard output in a fixed format, and stops after 20 packets have been captured.

Program Workflow

- Program Initialisation — include the required socket/header files; allocate a 65536-byte receive buffer; declare sockaddr_in structures for the source and destination addresses and a packet counter.
- Configuration — ROLL_NO and ASSIGNED_PROTOCOL are defined as compile-time macros; MAX_PACKETS is fixed at 20.
- Raw Socket Creation — socket(AF_INET, SOCK_RAW, IPPROTO_ICMP) is called; if creation fails (e.g. not run as root) an error is printed via perror() and the program exits.
- Packet Capture Loop — recvfrom() is called repeatedly inside a while loop until packet_count reaches MAX_PACKETS.
- Packet Processing — the IP header is read via a struct iphdr pointer into the buffer; header length is computed dynamically as iph->ihl * 4 rather than assumed to be 20 bytes; source and destination addresses, protocol number, TTL, total length and IP version are extracted.
- Protocol-Specific Parsing — since the protocol is ICMP, a struct icmphdr pointer is placed immediately after the IP header (buffer + iphlen) to read the ICMP Type and Code.
- Output — each field is printed to stdout in a fixed, delimited format per packet; the shell redirection (> program_output.txt) is what actually saves this to a file, the program itself does not open a file handle.
- Termination — once 20 packets have been captured, the raw socket is closed and the program exits.

5. Traffic Generation

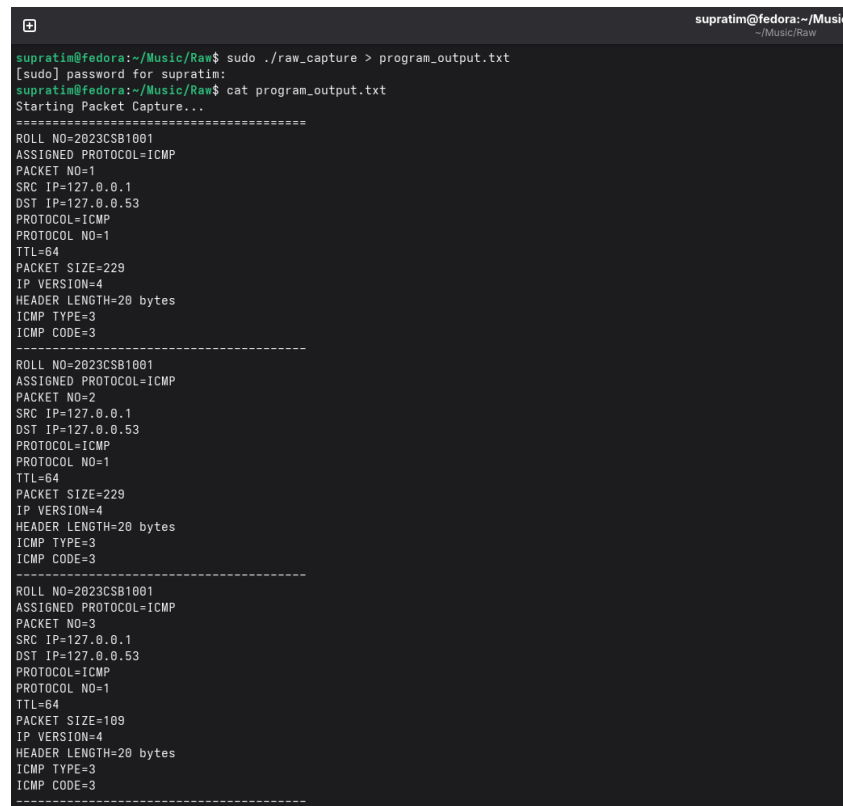
ICMP traffic was generated using the ping command, which sends ICMP Echo Request packets and listens for Echo Reply packets. Two runs were used while raw_capture and Wireshark were both listening:

```
ping -c 5 8.8.8.8
ping -c 25 8.8.8.8
```

Both commands target Google's public DNS server (8.8.8.8). The first, shorter run was used to confirm connectivity and round-trip times before starting the full 25-packet run alongside the capture.

6. Experimental Results

raw_capture successfully captured 20 ICMP packets, as fixed by MAX_PACKETS, and wrote the following fields for every packet: Source IP, Destination IP, Protocol, Protocol Number, TTL, Packet Size, IP Version, Header Length, ICMP Type and ICMP Code.



```
supratim@fedora:~/Music/Raw$ sudo ./raw_capture > program_output.txt
[sudo] password for supratim:
supratim@fedora:~/Music/Raw$ cat program_output.txt
Starting Packet Capture...
=====
ROLL NO=2023CSB1001
ASSIGNED PROTOCOL=ICMP
PACKET NO=1
SRC IP=127.0.0.1
DST IP=127.0.0.53
PROTOCOL=ICMP
PROTOCOL NO=1
TTL=64
PACKET SIZE=229
IP VERSION=4
HEADER LENGTH=20 bytes
ICMP TYPE=3
ICMP CODE=3
=====
ROLL NO=2023CSB1001
ASSIGNED PROTOCOL=ICMP
PACKET NO=2
SRC IP=127.0.0.1
DST IP=127.0.0.53
PROTOCOL=ICMP
PROTOCOL NO=1
TTL=64
PACKET SIZE=229
IP VERSION=4
HEADER LENGTH=20 bytes
ICMP TYPE=3
ICMP CODE=3
=====
ROLL NO=2023CSB1001
ASSIGNED PROTOCOL=ICMP
PACKET NO=3
SRC IP=127.0.0.1
DST IP=127.0.0.53
PROTOCOL=ICMP
PROTOCOL NO=1
TTL=64
PACKET SIZE=109
IP VERSION=4
HEADER LENGTH=20 bytes
ICMP TYPE=3
ICMP CODE=3
=====
```

Fig. 3 — program_output.txt as printed by the raw socket program

A closer look at program_output.txt shows the 20 captured packets fall into two distinct groups:

- Packets 10–19 (10 packets): Echo Reply packets from 8.8.8.8 to 10.207.165.168 — TTL 116, packet size 84 bytes, ICMP Type 0 / Code 0. These correspond to the replies from the ping -c 25 8.8.8.8 traffic.
- Packets 1–9 and 20 (11 packets): Destination Unreachable packets between 127.0.0.1 and 127.0.0.53 — TTL 64, ICMP Type 3 / Code 3 (Port Unreachable), with sizes ranging from 109 to 270 bytes.

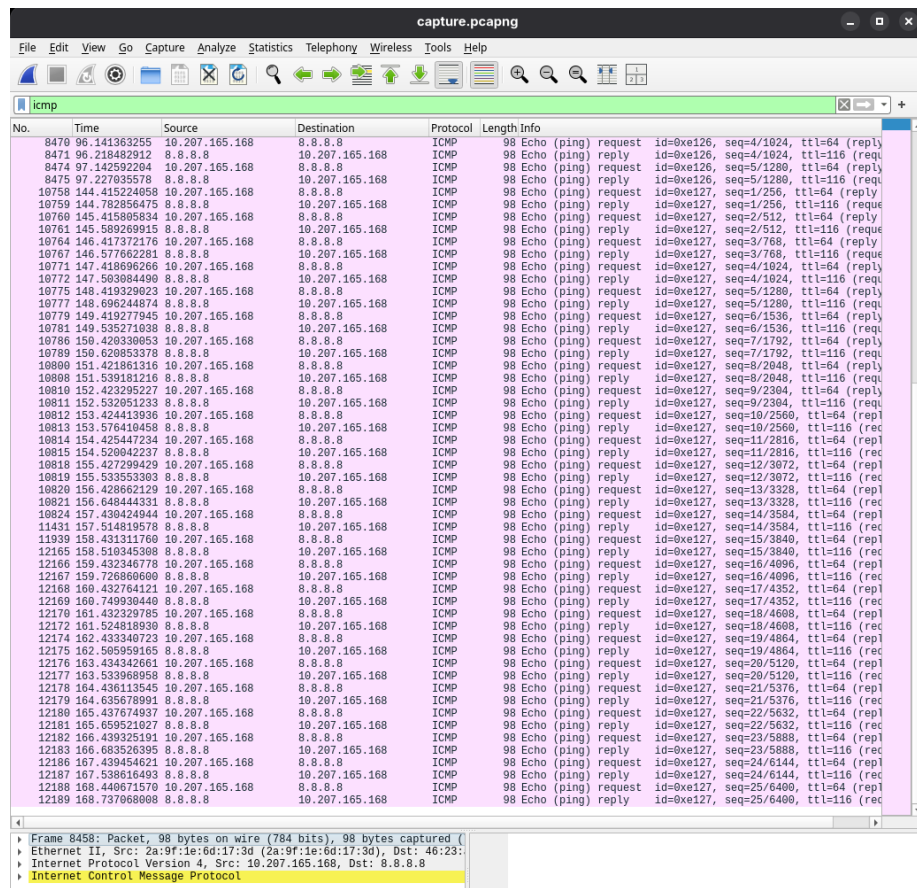
This is an interesting side-effect of using a raw ICMP socket without any address filtering: recvfrom() delivers every ICMP packet arriving on the host, not just packets to or from the target being pinged. The loopback traffic between 127.0.0.1 and the local DNS stub resolver (127.0.0.53) appears to come from unrelated background

DNS lookups happening on the system at the same time, and it was captured simply because it was also ICMP. As a result, only 10 of the 20 packets captured were actually part of the ping traffic that was intentionally generated for this assignment.

A second observation is that none of the 20 packets are outgoing Echo Request packets (Type 8) addressed to 8.8.8.8 — only the incoming Echo Replies were captured. This matches expected raw socket behaviour on Linux: a SOCK_RAW/IPPROTO_ICMP socket receives incoming ICMP traffic, but the Echo Requests generated by the ping utility itself are not looped back to the receiving raw socket in this setup, so they are visible in Wireshark but not in the program's own output.

7. Wireshark Verification

The same traffic was independently captured with Wireshark, run with `sudo` and filtered using the `icmp` display filter.



The image shows a Wireshark capture window titled 'capture.pcapng'. The display filter is set to 'icmp'. The packet list shows 20 packets, all of which are Echo (ping) replies from 8.8.8.8 to 10.207.165.168. The packet details pane shows the selected packet (No. 12189) as an Internet Control Message Protocol (ICMP) Echo (ping) reply. The packet bytes pane shows the raw data of the packet.

No.	Time	Source	Destination	Protocol	Length	Info
8470	96.141363255	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe126, seq=4/1024, ttl=64 (reply)
8471	96.218482912	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe126, seq=4/1024, ttl=116 (req)
8474	97.142592204	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe126, seq=5/1280, ttl=64 (reply)
8475	97.227835578	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe126, seq=5/1280, ttl=116 (req)
10750	144.415224058	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=1/256, ttl=64 (reply)
10759	144.782856475	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=1/256, ttl=116 (req)
10760	145.415805834	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=2/512, ttl=64 (reply)
10761	145.589269915	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=2/512, ttl=116 (req)
10764	146.417372176	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=3/768, ttl=64 (reply)
10767	146.577662281	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=3/768, ttl=116 (req)
10771	147.418696266	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=4/1024, ttl=64 (reply)
10772	147.503884490	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=4/1024, ttl=116 (req)
10775	148.419329023	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=5/1280, ttl=64 (reply)
10777	148.696244074	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=5/1280, ttl=116 (req)
10779	149.419277945	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=6/1536, ttl=64 (reply)
10781	149.535271038	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=6/1536, ttl=116 (req)
10786	150.426330853	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=7/1792, ttl=64 (reply)
10789	150.620853378	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=7/1792, ttl=116 (req)
10800	151.421861316	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=8/2048, ttl=64 (reply)
10808	151.539181216	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=8/2048, ttl=116 (req)
10810	152.423295227	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=9/2304, ttl=64 (reply)
10811	152.532051233	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=9/2304, ttl=116 (req)
10812	153.424413936	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=10/2560, ttl=64 (reply)
10813	153.576410458	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=10/2560, ttl=116 (req)
10814	154.425447234	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=11/2816, ttl=64 (reply)
10815	154.520042237	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=11/2816, ttl=116 (req)
10818	155.427299429	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=12/3072, ttl=64 (reply)
10819	155.533553863	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=12/3072, ttl=116 (req)
10820	156.428662129	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=13/3328, ttl=64 (reply)
10821	156.648444331	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=13/3328, ttl=116 (req)
10824	157.438424944	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=14/3584, ttl=64 (reply)
11431	157.514819578	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=14/3584, ttl=116 (req)
11039	158.431311760	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=15/3840, ttl=64 (reply)
12165	158.510345308	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=15/3840, ttl=116 (req)
12166	159.432346778	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=16/4096, ttl=64 (reply)
12167	159.726806060	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=16/4096, ttl=116 (req)
12168	160.432764121	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=17/4352, ttl=64 (reply)
12169	160.749390440	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=17/4352, ttl=116 (req)
12170	161.432329785	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=18/4608, ttl=64 (reply)
12172	161.524818939	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=18/4608, ttl=116 (req)
12174	162.433340723	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=19/4864, ttl=64 (reply)
12175	162.505959165	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=19/4864, ttl=116 (req)
12176	163.434342661	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=20/5120, ttl=64 (reply)
12177	163.533968958	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=20/5120, ttl=116 (req)
12178	164.436113545	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=21/5376, ttl=64 (reply)
12179	164.635678991	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=21/5376, ttl=116 (req)
12180	165.437674937	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=22/5632, ttl=64 (reply)
12181	165.659521027	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=22/5632, ttl=116 (req)
12182	166.439325191	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=23/5888, ttl=64 (reply)
12183	166.683525395	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=23/5888, ttl=116 (req)
12186	167.439454621	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=24/6144, ttl=64 (reply)
12187	167.538616493	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=24/6144, ttl=116 (req)
12188	168.440671570	10.207.165.168	8.8.8.8	ICMP	98	Echo (ping) request id=0xe127, seq=25/6400, ttl=64 (reply)
12189	168.737068008	8.8.8.8	10.207.165.168	ICMP	98	Echo (ping) reply id=0xe127, seq=25/6400, ttl=116 (req)

Fig. 4 — Wireshark capture.pcapng filtered on icmp, showing request/reply pairs between 10.207.165.168 and 8.8.8.8

The Wireshark capture confirms the round trip that raw_capture only saw one side of: each Echo Request (id 0xe126, 0xe127, ...) leaves 10.207.165.168 with TTL 64 and is answered by an Echo Reply from 8.8.8.8 with TTL 116 — the exact TTL value recorded for the Echo Reply packets in program_output.txt. The following fields matched between the two tools for every reply packet:

- Source and Destination IP addresses
- Protocol number (1 for ICMP)

- TTL
- Packet size
- ICMP Type and Code

This cross-check confirms that the header values extracted by the raw socket program are accurate, and also shows that Wireshark captured traffic (the outgoing requests) that the raw socket program itself did not report, as discussed above.

8. Header Analysis

IP Header Analysis

The IP header carries the information needed to route each packet across the network.

Header Field	Description	Observed Value (Packet 10, Echo Reply)
Source IP Address	IP address of the sender	8.8.8.8
Destination IP Address	IP address of the receiver	10.207.165.168
Version	Internet Protocol version	4 (IPv4)
Header Length	Length of the IP header	20 bytes
Protocol Number	Transport/ICMP layer identifier	1 (ICMP)
TTL (Time To Live)	Maximum number of hops allowed	116 (64 on the outgoing request, per Wireshark)
Packet Size	Total size of the packet	84 bytes

ICMP Header Analysis

For ICMP packets, the Type and Code fields were extracted directly after the IP header:

Header Field	Description
ICMP Type	Type of ICMP message
ICMP Code	Subtype of the ICMP message

Example — Echo Reply (packets 10–19):

- ICMP Type : 0 (Echo Reply)
- ICMP Code : 0

Example — Destination Unreachable (packets 1–9, 20):

- ICMP Type : 3 (Destination Unreachable)
- ICMP Code : 3 (Port Unreachable)

9. Program Enhancements

Beyond the basic requirement of printing Source IP, Destination IP, Protocol, TTL and Packet Size, the following enhancements were included:

- IP Version field added to the output.
- IP Header Length computed dynamically from the header itself (`iph->ihl * 4`) instead of being hard-coded, so the program stays correct even if a packet carries IP options.
- Clear, delimited per-packet output ("-----") with packet numbering, making the text file easy to scan or parse later.
- Basic error handling on socket creation and on `recvfrom()`, with `perror()` used to report the underlying system error (e.g. missing root privileges).
- Output saved to `program_output.txt` via shell redirection, and the same traffic independently verified in Wireshark and saved as `capture.pcapng`.

10. Reflection Questions

Why are root privileges required for raw sockets?

A raw socket lets a program read (and, if it chooses to build its own packets, write) IP-layer traffic directly, instead of going through the kernel's usual TCP/UDP handling. Because this gives a process visibility into traffic that isn't addressed to it, and the ability to craft packets that bypass normal protocol checks, Linux restricts `SOCK_RAW` to processes with root privileges (or the `CAP_NET_RAW` capability) to stop unprivileged programs from sniffing or spoofing traffic.

How are raw sockets different from TCP/UDP sockets?

A TCP or UDP socket operates at the transport layer: the kernel builds and parses the TCP/UDP and IP headers automatically, and the application only deals with a byte stream or datagrams. A raw socket instead exposes the network layer directly — the application receives the full IP packet (and, depending on the protocol, has to parse the transport-layer header itself, as this program does for ICMP). This gives far more visibility and control, at the cost of having to handle header parsing manually.

State one advantage and one limitation of raw sockets.

Advantage: raw sockets expose the complete packet, including header fields that a normal socket hides, which makes them well suited to packet analysis, custom monitoring tools and protocol research. Limitation: they require root privileges and considerably more manual work — the program has to parse IP and protocol headers by hand and handle platform-specific struct layouts — making them harder to use correctly than a standard TCP/UDP socket.

Mention one networking or cybersecurity application of raw sockets.

Raw sockets are the basis for network intrusion detection and traffic-analysis tools, which inspect packets at the IP layer to spot scanning activity, malformed packets, or other signs of an attack that would not be visible through a normal TCP/UDP socket.

11. Conclusion

This assignment implemented a raw socket ICMP packet capture program in C on Linux, generated live ICMP traffic with ping, and captured 20 packets whose header fields were extracted and logged to `program_output.txt`. Cross-checking the results against a parallel Wireshark capture confirmed that the IP and ICMP header values reported by the program — source/destination addresses, TTL, packet size, and ICMP

type/code — were accurate. The exercise also surfaced a couple of practical details that only show up once real traffic is captured: an unfiltered raw socket picks up unrelated ICMP traffic on the host (in this case, local destination-unreachable messages between 127.0.0.1 and 127.0.0.53), and it only sees the incoming side of the ping exchange rather than the outgoing Echo Requests. Overall, the assignment gave hands-on experience with raw socket programming, IP/ICMP header structures, and comparing a custom packet capture tool against Wireshark.